

PRODUCTION AI ENGINEERING · S2

REAL VS SIMULATED

Real vs simulated: what actually ran

Which parts of the S2 POC exercised the real mechanism, which are faithful local substitutes, what was recorded and what was injected on purpose.

S2

S2 · Production AI Engineering

3 min read · Published 2026-10-08 · Real vs simulated

Real

the actual mechanism was exercised.

- Retrieval: BM25 (k1 1.2, b 0.75) and vector search over real embeddings computed by `nomic-embed-text` through Ollama, fused by reciprocal rank fusion; every query embedding of the run is on its tape (`tape/query_embeddings.jsonl`).
- Admission: the recheck at the source, the scope, lifecycle, authority and conflict gates, the evidence packet and the assembler are the code under test, deterministic, with no model involved (experiments A, B, C and E at evidence level).
- Generation: every end-to-end answer was produced live by a local model through Ollama with structured output, 66 naive and 66 governed calls to gpt-oss:20b in experiment D, 22 and 22 to qwen3:8b in the sensitivity run, and the ablation and judge calls on their own tapes.
- Verification and binding: the lexical citation verifier and the binding rules ran on every recorded answer; experiment D2 compares them with qwen3:8b as a model judge on 40 held-out claim-citation pairs.
- Scoring: a scorer that is the only reader of the hand-written labels; `tests/test_isolation.py` fails if the system under test imports it.

Simulated

a faithful local substitute for an external system.

- Meridian Commerce itself: two tenants, their runbooks, change policies, tickets, release notes, wiki, restricted documents and vendor documents. Every document is synthetic and labelled so.
- The identity provider (four principals and their groups), each source's permission and status API, the nightly index connector and its 02:00 watermark, the release system and the CMDB.
- The clock: frozen at 10:30 on the incident day, with four source events between the watermark and the question.

Recorded

captured model traffic, reused for exact replay.

- Every model exchange of the run, request and response, with the server's token counts: `tape/model.jsonl` (primary), `tape/model-sensitivity.jsonl`, `tape/judge.jsonl` .

- Replay re-executes every experiment from the tapes with no model and no Ollama and compares every rows file byte for byte (`make replay`).

Injected

a failure introduced on purpose.

- One failure per case: a similar runbook for another service, a near-identical identifier, a stale version ranked high, another tenant's evidence, staging evidence, lower-authority guidance contradicting the runbook of record, an unresolved conflict between owners, a qualifier the budget is likely to cut, a four-source incident, history cited as the approved fix, an answer only in an unreadable document, revocation and supersession after the watermark, a poisoned page, insufficient evidence, a newest document that is not the authority; and three positive controls.
- Canary strings in restricted and revoked documents, so that leakage is a string match.
- One page carrying a synthetic, clearly marked injected instruction.

Architecture

design this POC does not exercise.

- Graph retrieval, rerankers, query rewriting and agentic retrieval (drawn as proposed or discussed, never built).
- A real identity provider, document store, CMDB or release system; production latency, load, batching and caching of the recheck.
- Any hosted model or service.

What the simulation idealises

- Every source answers the recheck immediately and consistently; a source that cannot answer is simulated as "unavailable", and the pipeline fails closed.
- The CMDB names a runbook of record for every service and procedure the cases need; a real CMDB has gaps, and §21 of the technical edition says what a gap does.
- Sections are clean and headed; a real corpus has scanned PDFs, tables and documents with no structure.
- 22 held-out cases, each with one planted failure; a real estate has failures nobody planted, in proportions nobody knows.

PRODUCTION AI ENGINEERING · THE SERIES SO FAR

FOUNDATION

F1 · MCP Tool Sprawl

Your AI agent has 500 MCP tools. Now what?

FOUNDATION

F2 · Layered Architecture

Your agent works in a demo. Why does it break in production?

FOUNDATION

F3 · Headless AI

Your AI shouldn't live inside the UI

STATE

S1 · Memory, Context & State

Your agent remembers everything. That's a problem.

TRUST

T2 · Authorization & Policy

What is this agent actually allowed to do?

TRUST

T3 · Human-in-the-Loop

Authorized. Should it still act?

TRUST

T5 · Observability & Governance

Your AI agent did something in production. Can you explain exactly what happened?

TRUST

T4 · AI Control Plane

Your agents shouldn't govern themselves

TRUST

T1 · Agent Identity

Who is acting, and on whose authority?

TRUST

T6 · Agent, Tool & MCP Security

Assume the context is hostile.

RELIABILITY

R1 + R2 · Evals, Observability & Reliability

"The agent failed" is not an operational signal.

CAPSTONE

P1 · The Agent Is Not the Architecture

The final reference architecture

COORDINATION

C1 · Multi-Agent & A2A

Do you actually need multiple agents?

OPERATIONS

O1 + O2 · Operating AI Agents at Scale

Cost, latency, scale and lifecycle

CURRENT

S2 · Enterprise Knowledge & RAG

Your agent found the right document. Why did it give the wrong answer?

The full learning map (Start Here)

S2 · Production AI Engineering

Production AI Engineering · S2. Every measured number here is substituted at build time from `enterprise_knowledge_rag_poc/runs/2026-10-08-helldout/facts.json`. Figures carry their provenance.

Medium edition Technical deep dive Evidence Check POC README QA report